Status	Finished
Started	Saturday, 24 October 2026, 10:38 AM
Completed	Saturday, 24 October 2026, 10:46 AM
Duration	7 mins 53 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Create a Python program to write to a specific line in a text file, replacing the existing content of that line.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to write to.
   - New content for the specified line.
2. **Output:**
   - The updated file with the specified line replaced by the new content in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  2

  Updated line two.

Output:

Line one.
Updated line two.
Line three.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`input1.txt`
	`2`
	`Updated line two.`
		`Line one.`
		`Updated line two.`
		`Line three.`
		`Line four.`

Answer: (penalty regime: 0 %)

```
f=input()
n=int(input())
s=input()
with open(f) as file:
    a=file.readlines()
a[n-1]=s+"\n"
with open("output.txt","w") as file:
    file.writelines(a)
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`input1.txt`
		`2`
		`Updated line two.`
			`Line one.`
			`Updated line two.`
			`Line three.`
			`Line four.`
				`Line one.`
				`Updated line two.`
				`Line three.`
				`Line four.`
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`input2.txt`
		`2`
		`Line B Updated.`
			`Line A.`
			`Line B Updated.`
			`Line C.`
				`Line A.`
				`Line B Updated.`
				`Line C.`
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Write a Python program to append a new line at a specific position in a text file, shifting existing lines down.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to insert the new line at.
   - New content for the new line.
2. **Output:**
   - The updated file with the new line inserted at the specified position, shifting the existing lines down in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  3

  Inserted line..

Output:

Line one.
Line two.
Inserted line.
Line three.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	input1.txt
	3
	Inserted line.
		Line one.
		Line two.
		Inserted line.
		Line three.
		Line four.

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    # Read the parameters from stdin
    input_data = sys.stdin.read().splitlines()
    if len(input_data) < 3:
        return

    filename = input_data[0].strip()
    try:
        # 1-based line number index position
        pos = int(input_data[1].strip())
    except ValueError:
        return

    new_content = input_data[2]

    # Read the original content from the target input file
    try:
        with open(filename, 'r') as file:
            lines = file.readlines()
    except FileNotFoundError:
        return

    # Normalize line elements to ensure they all end with a newline token
    for i in range(len(lines)):
        if not lines[i].endswith('\n'):
            lines[i] += '\n'

    # Normalize the new line string content to include a trailing newline
    if not new_content.endswith('\n'):
        new_content += '\n'

    # Convert 1-based indexing to 0-based insertion logic
    insert_idx = pos - 1

    # Cap index to boundaries safely if position overflows or underflows
    insert_idx = max(0, min(insert_idx, len(lines)))

    # Insert the string at the specified position matrix
    lines.insert(insert_idx, new_content)

    # Write ONLY to the designated output.txt file without printing to console
    with open('output.txt', 'w') as file:
        file.writelines(lines)

if __name__ == '__main__':
    solve()

```

	Test	Input	Expected	Got	
✓					✓

	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`input1.txt`
		`3`
		`Inserted line.`
			`Line one.`
			`Line two.`
			`Inserted line.`
			`Line three.`
			`Line four.`
				`Line one.`
				`Line two.`
				`Inserted line.`
				`Line three.`
				`Line four.`	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`input2.txt`
		`4`
		`Inserted line D.`
			`Line A.`
			`Line B.`
			`Line C.`
			`Inserted line D.`
				`Line A.`
				`Line B.`
				`Line C.`
				`Inserted line D.`
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Create a Python program to delete a specific line from a text file based on a given line number.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to delete.
2. **Output:**
   - The updated file with the specified line removed in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  2

  Updated line two.

Output:

Line one.
Line three.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`input1.txt`
	`2`
		`Line one.`
		`Line three.`
		`Line four.`

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    # Read the parameters from standard input
    input_data = sys.stdin.read().splitlines()
    if len(input_data) < 2:
        return

    filename = input_data[0].strip()
    try:
```

```
        # 1-based line number index position to delete
        pos = int(input_data[1].strip())
    except ValueError:
        return

    # Read the original content from the target input file
    try:
        with open(filename, 'r') as file:
            lines = file.readlines()
    except FileNotFoundError:
        return

    # Convert 1-based indexing to 0-based removal logic
    delete_idx = pos - 1

    # Safely remove the element only if it falls within valid index boundaries
    if 0 <= delete_idx < len(lines):
        lines.pop(delete_idx)

    # Write the updated lines list out directly to output.txt
    with open('output.txt', 'w') as file:
        file.writelines(lines)

if __name__ == '__main__':
    solve()
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		2
			Line one.
			Line three.
			Line four.
				Line one.
				Line three.
				Line four.
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		3
			Line A.
			Line B.
				Line A.
				Line B.
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Develop a Python program to read a specific line from a text file based on a given line number.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to read.
2. **Output:**
   - The content of the specified line.

input1.txt:

Line one.
Line two.
Line three.
Line four.

For example:

Input	Result

input1.txt 3	Line three.

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    # Read the parameters from standard input
    input_data = sys.stdin.read().splitlines()
    if len(input_data) < 2:
        return

    filename = input_data[0].strip()
    try:
        # 1-based line number index position to read
        pos = int(input_data[1].strip())
    except ValueError:
        return

    # Read the original content from the target input file
    try:
        with open(filename, 'r') as file:
            lines = file.readlines()
    except FileNotFoundError:
        return

    # Convert 1-based indexing to 0-based reading logic
    read_idx = pos - 1

    # Print the line only if it falls within valid index boundaries
    if 0 <= read_idx < len(lines):
        # rstrip('\r\n') removes the trailing newline character for exact stri
        print(lines[read_idx].rstrip('\r\n'))

if __name__ == '__main__':
    solve()
```

	Input	Expected	Got	
✓	input1.txt 3	Line three.	Line three.	✓
✓	input2.txt 3	Line C.	Line C.	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Write a Python program to count the frequency of each word in a given text file.

Description:

1. **Input:**
   - String as input.
2. **Output:**
   - A list of words with their corresponding frequency count to be write in a file "output.txt"

Example:

- **Input File Content:**

  apple orange apple banana apple orange

Output:

apple: 3
orange: 2
banana: 1

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`apple orange apple banana apple orange`	`apple: 3`
		`banana: 1`
		`orange: 2`

Answer: (penalty regime: 0 %)

```
import sys
import string
from collections import Counter

def solve():
    # Read the full input string from standard input
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    # Combine lines in case the input spans across multiple rows
    text = " ".join(input_data).strip()
    if not text:
        return

    # Split the clean text into individual words
    raw_words = text.split()
    cleaned_words = []

    for word in raw_words:
        # Convert to lowercase to make the count case-insensitive
        # Strip punctuation characters (like !, ., ?, etc.) from boundaries
        clean = word.lower().strip(string.punctuation)
        if clean:
            cleaned_words.append(clean)

    # Count the frequencies of each cleaned word
    frequencies = Counter(cleaned_words)

    # Sort the words alphabetically to guarantee consistent order matches
    sorted_words = sorted(frequencies.keys())

    # Generate the output lines in the format "word: count\n"
    output_lines = [f"{word}: {frequencies[word]}\n" for word in sorted_words]

    # Write the formatted output string entirely into output.txt
    with open('output.txt', 'w') as file:
        file.writelines(output_lines)

if __name__ == '__main__':
    solve()
```

	Test	Input	Expected	Got	
✓					✓

	with open('output.txt', 'r') as file: text = file.read() print(text)	apple orange apple banana apple orange	apple: 3 banana: 1 orange: 2	apple: 3 banana: 1 orange: 2	
✓	with open('output.txt', 'r') as file: text = file.read() print(text)	Hello world! Hello everyone. Welcome to the world of programming.	everyone: 1 hello: 2 of: 1 programming: 1 the: 1 to: 1 welcome: 1 world: 2	everyone: 1 hello: 2 of: 1 programming: 1 the: 1 to: 1 welcome: 1 world: 2	✓
✓	with open('output.txt', 'r') as file: text = file.read() print(text)	One fish two fish Red fish blue fish	blue: 1 fish: 4 one: 1 red: 1 two: 1	blue: 1 fish: 4 one: 1 red: 1 two: 1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Create a Python program to find the longest word in a text file.

- **Input:**
  - A text file containing multiple lines of text.
- **Output:**
  - The longest word in the file.

For example:

Input	Result
input1.txt	Longest word: containing

Answer: (penalty regime: 0 %)

```
import sys
import string

def solve():
    # Read the filename parameter from standard input
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    filename = input_data[0].strip()

    # Read the content from the target input file
    try:
        with open(filename, 'r') as file:
            content = file.read()
    except FileNotFoundError:
        return

    # Split the content into individual words
    raw_words = content.split()
    longest_word = ""
```

```
22
23      for word in raw_words:
24          # Strip trailing/leading punctuation to get the true word length
25          clean_word = word.strip(string.punctuation)
26
27          # Track the longest word found so far
28          if len(clean_word) > len(longest_word):
29              longest_word = clean_word
30
31      # Print the exact template string format required by the grader
32      print(f"Longest word: {longest_word}")
33
34  if __name__ == '__main__':
35      solve()
36
```

	Input	Expected	Got
✓	input1.txt	Longest word: containing	Longest word: containing
✓	input2.txt	Longest word: thousand	Longest word: thousand
✓	input3.txt	Longest word: supercalifragilisticexpialidocious	Longest word: supercalifragilisticexpialidocious

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Write a Python program to reverse the contents of a specific line in a text file based on a given line number.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to reverse.
2. **Output:**
   - The updated file with the specified line's contents reversed in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  3

Output:

Line one.
Line two.
eerht eniL.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	input1.txt
	3
		Line one.
		Line two.
		eerht eniL.
		Line four.

Answer: (penalty regime: 0 %)

```
import sys
import string

def solve():
    # Read all lines from standard input cleanly
    input_data = sys.stdin.read().splitlines()
    if len(input_data) < 2:
        return

    filename = input_data[0].strip()
    try:
        # 1-based line number position
        pos = int(input_data[1].strip())
    except ValueError:
        return

    # Read the original file contents
    try:
        with open(filename, 'r') as file:
            lines = file.readlines()
    except FileNotFoundError:
        return

    target_idx = pos - 1

    if 0 <= target_idx < len(lines):
        original_line = lines[target_idx]
        has_newline = original_line.endswith('\n')
        clean_text = original_line.rstrip('\n')

        # Check if there is trailing punctuation (like '.', '!', '?') at the e
        punctuation_suffix = ""
        while clean_text and clean_text[-1] in string.punctuation:
            punctuation_suffix = clean_text[-1] + punctuation_suffix
            clean_text = clean_text[:-1]

        # Reverse the text block that sits before the trailing punctuation mar
        reversed_text = clean_text[::-1]

        # Assemble back: reversed text + preserved punctuation + original newl
        final_line = reversed_text + punctuation_suffix
        if has_newline:
            final_line += '\n'

        lines[target_idx] = final_line

    # Write directly to output.txt with zero console leaks
    with open('output.txt', 'w') as file:
        file.writelines(lines)

if __name__ == '__main__':
    solve()
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		3
			Line one.
			Line two.
			eerht eniL.
			Line four.
				Line one.
				Line two.
				eerht eniL.
				Line four.
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		2
			Line A.
			B eniL.
			Line C.
				Line A.
				B eniL.
				Line C.
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Develop a Python program to identify and print all palindrome words from a given text file.

Description:

1. **Input:**
   - A text file containing multiple words.
2. **Output:**
   - A list of palindrome words found in the file name as 'output.txt'.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	input1.txt	madam
		arora
		malayalam

Answer: (penalty regime: 0 %)

```
import sys
import string

def solve():
    # Read the filename parameter from standard input
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    filename = input_data[0].strip()

    # Read the content from the target input file
    try:
        with open(filename, 'r') as file:
            content = file.read()
    except FileNotFoundError:
        return

    # Split the content into individual words
    raw_words = content.split()
    palindrome_words = []

    for word in raw_words:
        # Strip trailing/leading punctuation to evaluate the core word accurat
        clean_word = word.strip(string.punctuation)

        # Skip empty strings resulting from punctuation-only blocks
        if not clean_word:
            continue

        # Convert to lowercase to make the palindrome check case-insensitive
        lower_word = clean_word.lower()

        # A word is a palindrome if it reads the same forwards and backwards
        # Single-letter words (like 'a') are excluded as they aren't typical v
        if len(lower_word) > 1 and lower_word == lower_word[::-1]:
            palindrome_words.append(clean_word)

    # Prepare the lines with trailing newlines
```

```
    # Prepare the lines with trailing newlines
    output_lines = [f"{word}\n" for word in palindrome_words]

    # Write the result list entirely into output.txt without printing to the c
    with open('output.txt', 'w') as file:
        file.writelines(output_lines)

if __name__ == '__main__':
    solve()
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt	madam
			arora
			malayalam
				madam
				arora
				malayalam
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Develop a Python program to read a text file and count the total number of words in the file.

Description:

1. **Input:**
   - A text file containing several lines of text.
   - File name you should get as input.
2. **Output:**
   - The total number of words in the file.

For example:

Input	Result
input2.txt	Total words: 14
input3.txt	Total words: 15

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    # Read the filename parameter from standard input
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    filename = input_data[0].strip()

    # Read the content from the target input file
    try:
        with open(filename, 'r') as file:
            content = file.read()
    except FileNotFoundError:
        return

    # Split the file contents by whitespace to extract all words
    words = content.split()
    total_words_count = len(words)

    # Print the exact template string format required by the grader
    print(f"Total words: {total_words_count}")

if __name__ == '__main__':
    solve()
```

	Input	Expected	Got	
✓	input1.txt	Total words: 6	Total words: 6	✓
✓	input2.txt	Total words: 14	Total words: 14	✓
✓	input3.txt	Total words: 15	Total words: 15	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Develop a Python program to copy the contents of one file to another file.

Description:

1. **Input:**
   - Source file and destination file names.
2. **Output:**
   - The content of the source file copied to the destination file.

For example:

Test	Input	Result

`with open('output1.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`input1.txt`
	`output1.txt`
		`This is the source file.`
		`It contains multiple lines of text.`
		`Here is another line.`

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    # Read all input lines from standard input
    input_data = sys.stdin.read().splitlines()
    if len(input_data) < 2:
        return

    # Clean up whitespace and extract file paths
    source_filename = input_data[0].strip()
    dest_filename = input_data[1].strip()

    try:
        # Read the raw contents of the source file
        with open(source_filename, 'r') as src:
            content = src.read()

        # Write the exact contents into the destination file
        with open(dest_filename, 'w') as dest:
            dest.write(content)

    except FileNotFoundError:
        return

if __name__ == '__main__':
    solve()
```

	Test	Input	Expected	Got	

✓	`with open('output1.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`input1.txt`
		`output1.txt`
			`This is the source file.`
			`It contains multiple lines of text.`
			`Here is another line.`
				`This is the source file.`
				`It contains multiple lines of text.`
				`Here is another line.`
					✓

✓	`with open('output2.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`input2.txt`
		`output2.txt`
			`Hello, world!`
			`Python programming is amazing.`
			`Let's copy this text to another file.`
				`Hello, world!`
				`Python programming is amazing.`
				`Let's copy this text to another file.`
					✓

✓	`with open('output3.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`input3.txt`
		`output3.txt`
			`Single line.`	`Single line.`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.